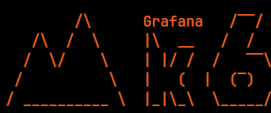


Задание 1

```
1 import http from 'k6/http'
2 import { check, sleep } from 'k6'
3
4 export const options = {
5   vus: 10,
6   duration: '30s',
7   summaryTrendStats: ['avg', 'min', 'med', 'max', 'p(50)', 'p(95)', 'p(99)']
8 }
9
10
11 export default function() {
12   const res = http.get('http://localhost:8080')
13
14   check(res, {
15     'status is 200': (r) => r.status === 200,
16   })
17
18   sleep(1)
19 }
```

test-constant.json

```
lly@lly ~/s/h/l/t/1 (master)> k6 run test-constant.js
```



```

  execution: local
    script: test-constant.js
    output: -

  scenarios: (100.00%) 1 scenario, 10 max VUs, 1m0s max duration (incl. graceful stop):
    * default: 10 looping VUs for 30s (gracefulStop: 30s)

  TOTAL RESULTS

  checks_total.....: 300      9.9827/s
  checks_succeeded...: 100.00% 300 out of 300
  checks_failed.....: 0.00%   0 out of 300

  ✓ status is 200

  HTTP
  http_req_duration.....: avg=1.11ms min=415.75µs med=895.42µs max=7.97ms p(50)=895.42µs p(95)=1.14ms p(99)=7.48ms
    { expected_response:true }...: avg=1.11ms min=415.75µs med=895.42µs max=7.97ms p(50)=895.42µs p(95)=1.14ms p(99)=7.48ms
  http_req_failed.....: 0.00%   0 out of 300
  http_reqs.....: 300      9.9827/s
```

```
EXECUTION
iteration_duration.....: avg=1s    min=1s    med=1s    max=1s    p(50)=1s    p(95)=1s    p(99)=1s
iterations.....: 300    9.9827/s
vus.....: 10    min=10    max=10
vus_max.....: 10    min=10    max=10

NETWORK
data_received.....: 340 kB 11 kB/s
data_sent.....: 21 kB 699 B/s

running (0m30.1s), 00/10 VUs, 300 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs 30s
```

k6 run [test-constant.js](#)

rps = 9.9827/s
p50 = 895.42µs
p(95)=1.14ms
p(99)=7.48ms
checks_failed.....: 0.00% 0 out of 300

Метрика	Значение	Интерпретация
http_reqs	300	кол-во выполненных запросов
http_req_duration{avg}	1.11ms	Среднее время запроса
http_req_duration{p95}	1.14ms	95% запросов быстрее 1.14ms
http_req_duration{p99}	7.48ms	99% запросов быстрее 7.48ms
http_req_failed	0.00%	нет ошибок

Задание 2

```
1 import http from 'k6/http'
2 import { check, sleep } from 'k6'
3
4 export const options = {
5   stages: [
6     { duration: '30s', target: 10 },
7   ],
8   summaryTrendStats: ['avg', 'min', 'med', 'max', 'p(50)', 'p(90)', 'p(95)', 'p(99)'],
9 }
10
11 export default function() {
12   const res = http.get('http://localhost:8080')
13
14   check(res, {
15     'status is 200': (r) => r.status === 200,
16   })
17
18   sleep(1)
19 }
```

[test-constant.js](#)

```
lly@lly ~/s/h/L/t/2 (master)> k6 run test-constant.js

      Grafana
    /-----\
   /         \
  /           \
 /             \
/               \
\               /
 \             /
  \           /
   \         /
    \-----/

execution: local
script: test-constant.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 1m0s max duration (incl. graceful stop):
 * default: Up to 10 looping VUs for 30s over 1 stages (gracefulRampDown: 30s, gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 153      4.983644/s
checks_succeeded...: 100.00% 153 out of 153
checks_failed.....: 0.00%   0 out of 153

✓ status is 200

HTTP
http_req_duration.....: avg=813.5µs min=375.36µs med=603.45µs max=5.47ms p(50)=603.45µs p(90)=701.96µs p(95)=4.44ms p(99)=4.94ms
{ expected_response:true }...: avg=813.5µs min=375.36µs med=603.45µs max=5.47ms p(50)=603.45µs p(90)=701.96µs p(95)=4.44ms p(99)=4.94ms
http_req_failed.....: 0.00% 0 out of 153
http_reqs.....: 153      4.983644/s
```

```
EXECUTION
iteration_duration.....: avg=1s      min=1s      med=1s      max=1s      p(50)=1s      p(90)=1s      p(95)=1s      p(99)=1s
iterations.....: 153      4.983644/s
vus.....: 9      min=1      max=9
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 174 kB 5.7 kB/s
data_sent.....: 11 kB 349 B/s

running (0m30.7s), 00/10 VUs, 153 complete and 0 interrupted iterations
default ✓ [=====] 00/10 VUs 30s
```

10 vus

```
lly@lly ~/s/h/l/t/2 (master)> k6 run test-constant.js

Grafana

execution: local
script: test-constant.js
output: -

scenarios: (100.00%) 1 scenario, 200 max VUs, 1m0s max duration (incl. graceful stop):
* default: Up to 200 looping VUs for 30s over 1 stages (gracefulRampDown: 30s, gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 3094    99.820489/s
checks_succeeded...: 100.00% 3094 out of 3094
checks_failed.....: 0.00%   0 out of 3094

✓ status is 200

HTTP
http_req_duration.....: avg=848.27µs min=328.65µs med=525.64µs max=15.69ms p(50)=525.64µs p(90)=871.33µs p(95)=4.72ms p(99)=5.51ms
  { expected_response:true }...: avg=848.27µs min=328.65µs med=525.64µs max=15.69ms p(50)=525.64µs p(90)=871.33µs p(95)=4.72ms p(99)=5.51ms
http_req_failed.....: 0.00%   0 out of 3094
http_reqs.....: 3094    99.820489/s

EXECUTION
iteration_duration.....: avg=1s    min=1s    med=1s    max=1.01s  p(50)=1s    p(90)=1s    p(95)=1s    p(99)=1s
iterations.....: 3094    99.820489/s
vus.....: 2    min=2    max=199
vus_max.....: 200    min=200    max=200

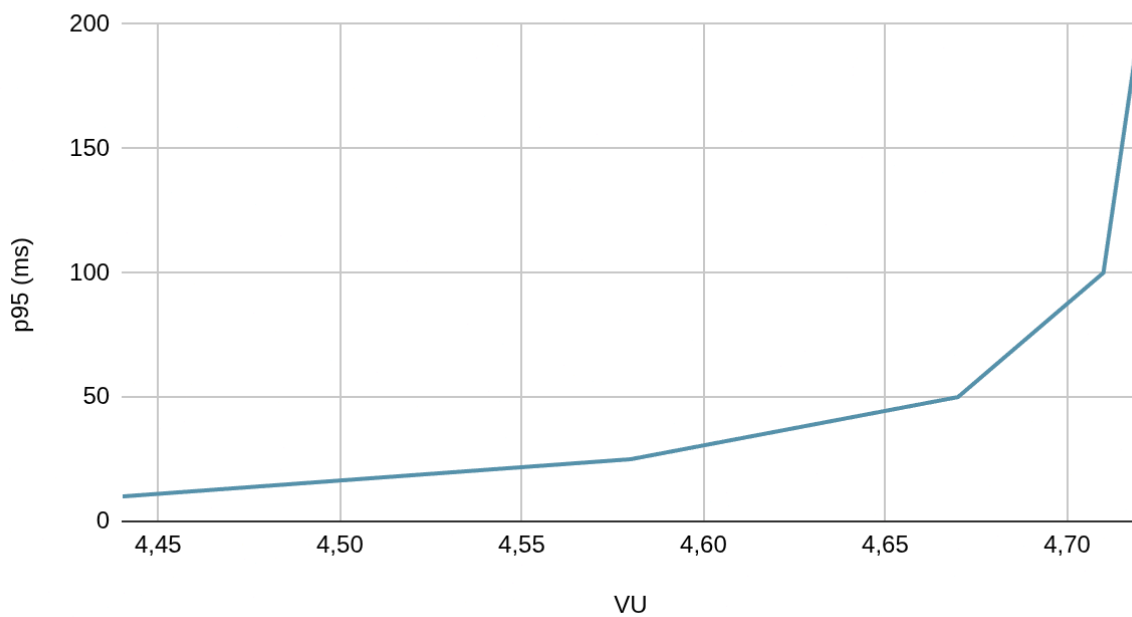
NETWORK
data_received.....: 3.5 MB 113 kB/s
data_sent.....: 217 kB 7.0 kB/s

running (0m31.0s), 000/200 VUs, 3094 complete and 0 interrupted iterations
default ✓ [=====] 000/200 VUs 30s
```

200vus

VU	RPS	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Errors (%)
10	4.983644/s	=603.45µs	=701.96µs	=4.44ms	=4.94ms	0.00%
25	12.472825/s	=524.42µs	=779.87µs	=4.58ms	=5.21ms	0.00%
50	24.9675/s	=650.62µs	=1.03ms	=4.67ms	=5.5ms	0.00%
100	49.910629/s	=611.66µs	=987.79µs	=4.71ms	=5.64ms	0.00%
200	99.820489/s	=525.64µs	=871.33µs	=4.72ms	=5.51ms	0.00%

p95 (ms) относительно параметра "VU"



RPS росло линейно, время отклика практически не менялось, для p50 0.5-0.65, p90 0.7-1, p95 4.4-4.7, p99 5-5.6, ошибки не появились.

Задание 3

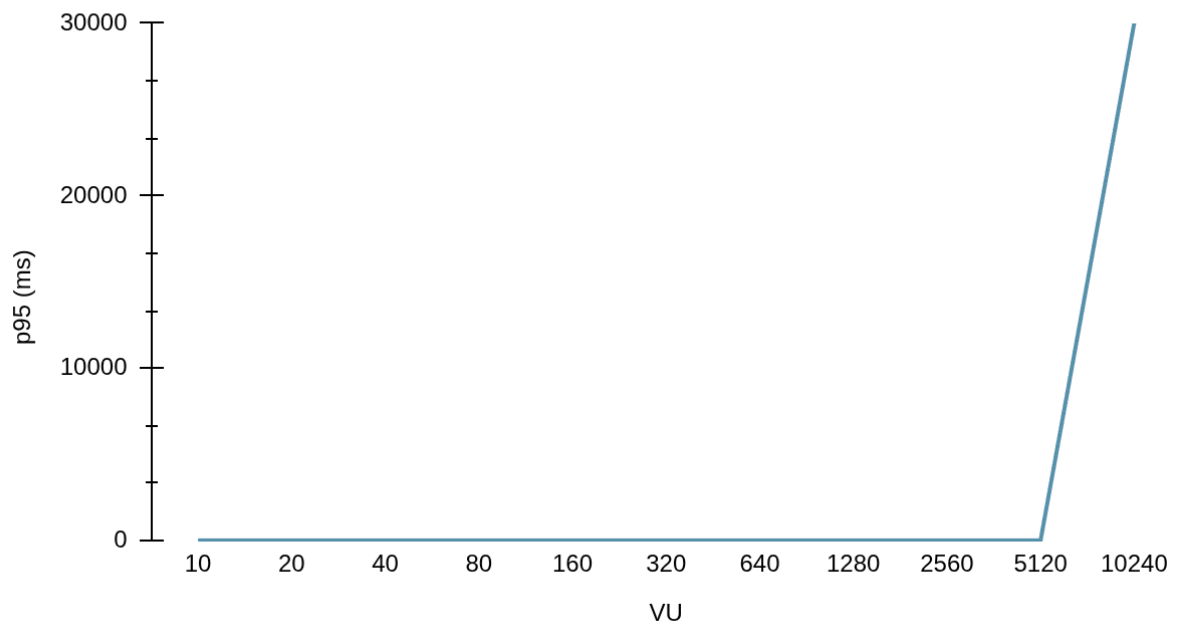
```
1 import http from 'k6/http'
2 import { check, sleep } from 'k6'
3
4 export const options = {
5   stages: [
6     { duration: '10s', target: 10 },
7   ],
8 }
9
10 export default function() {
11   const res = http.get('http://localhost:8080')
12
13   check(res, {
14     'status is 200': (r) => r.status === 200,
15   })
16
17   sleep(1)
18 }
```

[test-constant.js](#)

vus	rps	errors	p95
10	4.955612/s	0.00%	=4.73ms
20	9.948675/s	0.00%	=4.97ms
40	19.941744/s	0.00%	=5.03ms
80	39.92743/s	0.00%	=4.95ms
160	79.847081/s	0.00%	=4.92ms
320	159.7191/s	0.00%	=4.95ms
640	319.121095/s	0.00%	=5.21ms
1280	639.067133/s	0.00%	=5.14ms
2560	1276.927832/s	0.00%	=5.23ms
5120	2553.944138/s	0.00%	=5.71ms
10240	648.587643/s	15.33%	=30s

Причина деградации: сервер не справился с нагрузкой

p95 (ms) относительно параметра "VU"



До падения серверов ошибок не было, grps ~удваивался, p95 было предельно одинаково 4.73-5.71

Контрольные вопросы

1. Что такое нагрузочное тестирование и чем оно отличается от функционального? - нагрузочное тестирование проверяет как система работает, например, при большом количестве одновременных пользователей, а функциональное тестирование проверяет все ли работает так, как должно

2. Какие метрики производительности вы знаете? Определения RPS, latency, перцентилей. - latency, rps, error%, cpu%, mem%, network, disk.
RPS - количество запросов в секунду
latency - задержка
перцентиль - значение, быстрее которого выполнялся % запросов

3. Почему среднее не является достаточной метрикой? Пример. - потому что на среднее влияет самое большое и самое значение, например, 9 из 10 запросов выполнились за 1мс, а последний за 2012013021с, то среднее будет не объективным показателем

4. Что означают p50, p95, p99? Как интерпретировать p99 = 500 мс? - pN означает процент запросов, которые выполнились быстрее какого то времени, p99 = 500мс означает, что 99 процентов запросов выполнились быстрее 500мс

5. Как работает k6? Что такое Virtual User? - k6 запускает заданное число виртуальных пользователей на заданное количество времени, собирает метрики и в конце выводит результаты. Virtual User - имитация пользователя

6. Как определить точку деградации? Какие признаки указывают на деградацию. - определить точку деградации можно сравнив два вывода k6 для разного количества пользователей. Признаки: rps перестал расти (прирост <10% при удвоении VU), появились ошибки (>0.1%), p95 latency вырос в более, чем в два раза

7. Что такое thresholds в k6 и зачем они нужны? - thresholds - условия для метрик, если условие не выполнится, то k6 завершится с ошибкой

8. Как сформулировать корректное техническое требование к производительности? - требование должно содержать конкретные показатели метрик, то есть время ответа при n пользователях не должно превышать t ms

9. Почему при увеличении нагрузки RPS может перестать расти? - потому что система не успевает обработать такое большое количество запросов, например, ей не хватает ресурсов

10. Какие есть способы оптимизации производительности, если система не справляется с нагрузкой? - горизонтальное/вертикальное масштабирование, асинхронная обработка, кэширование